

CS3014 - Estructura de Datos Avanzados

Persistencia

Semana 3

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

25 de agosto de 2026

¿Qué aprenderemos hoy?



¿Qué aprenderemos hoy?



- ▶ La máquina de punteros: nuestro modelo de cómputo

¿Qué aprenderemos hoy?



- ▶ La máquina de punteros: nuestro modelo de cómputo
- ▶ Persistencia: parcial, total, confluyente y funcional

¿Qué aprenderemos hoy?



- ▶ La máquina de punteros: nuestro modelo de cómputo
- ▶ Persistencia: parcial, total, confluyente y funcional
- ▶ Path Copying en la práctica: Stack, Segment Tree y Trie

Máquina de Punteros

¿Por qué necesitamos un modelo de cómputo?



El problema

Para poder decir cosas como “esta estructura se puede volver persistente en $O(1)$ amortizado”, necesitamos primero acordar **qué operaciones cuestan $O(1)$** en la estructura original.

¿Por qué necesitamos un modelo de cómputo?



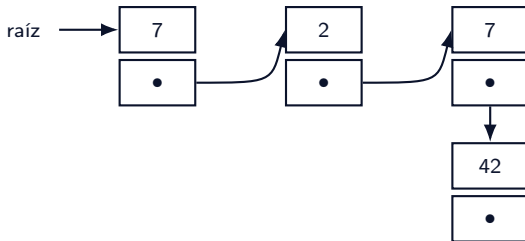
El problema

Para poder decir cosas como “esta estructura se puede volver persistente en $O(1)$ amortizado”, necesitamos primero acordar **qué operaciones cuestan $O(1)$** en la estructura original.

La respuesta de este curso: la máquina de punteros

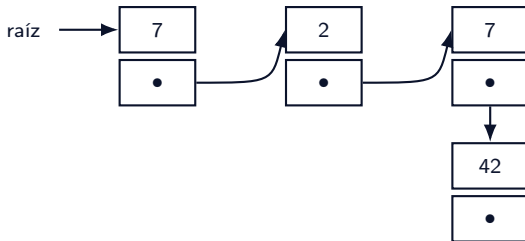
Un modelo simple, pero suficientemente general para describir casi cualquier estructura enlazada (listas, árboles, heaps, tries, ...).

La máquina de punteros (*pointer machine*)



Reglas del modelo

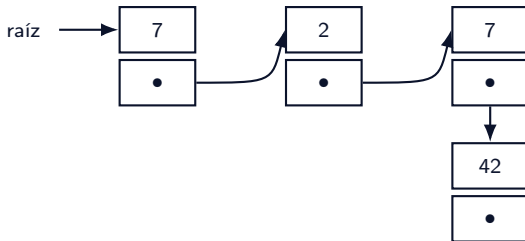
La máquina de punteros (*pointer machine*)



Reglas del modelo

- ▶ Cada nodo tiene $O(1)$ campos (datos o punteros a otros nodos).

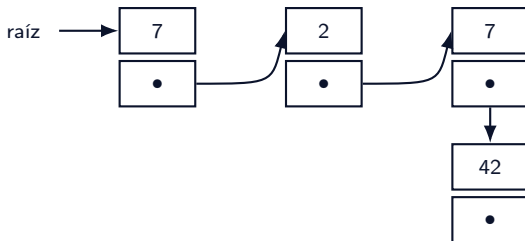
La máquina de punteros (*pointer machine*)



Reglas del modelo

- ▶ Cada nodo tiene $O(1)$ campos (datos o punteros a otros nodos).
- ▶ Hay un único nodo raíz, punto de entrada a toda la estructura.

La máquina de punteros (*pointer machine*)



Reglas del modelo

- ▶ Cada nodo tiene $O(1)$ campos (datos o punteros a otros nodos).
- ▶ Hay un único nodo raíz, punto de entrada a toda la estructura.
- ▶ Cada operación (leer/escribir un campo, seguir un puntero) cuesta $O(1)$.

Persistencia

¿Qué significa que una estructura sea persistente?



Definición

Una estructura de datos es **persistente** si, al modificarla, **conserva todas sus versiones anteriores**: cada operación se realiza sobre una **versión específica**, y una actualización crea (y devuelve) una **versión nueva** sin destruir la anterior.

¿Qué significa que una estructura sea persistente?



Definición

Una estructura de datos es **persistente** si, al modificarla, **conserva todas sus versiones anteriores**: cada operación se realiza sobre una **versión específica**, y una actualización crea (y devuelve) una **versión nueva** sin destruir la anterior.

Contraste

En una estructura **efímera** (lo normal), actualizar *destruye* el estado anterior: solo existe “el ahora”. La persistencia agrega una dimensión de **tiempo** a la estructura.

Niveles de persistencia: parcial y total



Persistencia parcial

Solo se puede **actualizar la versión más reciente**; se puede **consultar** cualquier versión anterior. Las versiones quedan ordenadas linealmente, como una línea de tiempo.

Niveles de persistencia: parcial y total



Persistencia parcial

Solo se puede **actualizar la versión más reciente**; se puede **consultar** cualquier versión anterior. Las versiones quedan ordenadas linealmente, como una línea de tiempo.

Persistencia total (*full*)

Se puede **actualizar cualquier versión**, no solo la última — cada actualización crea una nueva versión **hija** de la que se modificó. Las versiones forman un **árbol**.

Niveles de persistencia: confluyente y funcional



Persistencia confluyente

Además de actualizar, se puede **combinar (mezclar)** dos versiones distintas en una nueva. Las versiones forman un **DAG**, no un árbol.

Niveles de persistencia: confluyente y funcional



Persistencia confluyente

Además de actualizar, se puede **combinar (mezclar)** dos versiones distintas en una nueva. Las versiones forman un **DAG**, no un árbol.

Persistencia funcional

Una **técnica** de implementación (no un nivel más): la estructura **nunca modifica un nodo existente**, solo crea nodos nuevos. Cualquier nivel de persistencia se puede lograr así, típicamente al costo de un poco más de espacio.

Niveles de persistencia: confluyente y funcional



Persistencia confluyente

Además de actualizar, se puede **combinar (mezclar)** dos versiones distintas en una nueva. Las versiones forman un **DAG**, no un árbol.

Persistencia funcional

Una **técnica** de implementación (no un nivel más): la estructura **nunca modifica un nodo existente**, solo crea nodos nuevos. Cualquier nivel de persistencia se puede lograr así, típicamente al costo de un poco más de espacio.

Hoy nos enfocamos en **parcial** y **total** (las más usadas en la práctica), y usamos la idea **funcional** como base de la última sección.

El resultado general



Teorema (Driscoll, Sarnak, Sleator, Tarjan, 1989)

Cualquier estructura de datos sobre la máquina de punteros en la que a lo mucho $p = O(1)$ punteros apuntan a un mismo nodo (en cualquier versión) se puede volver **parcialmente persistente** con $O(1)$ **overhead multiplicativo amortizado** en tiempo, y $O(1)$ **espacio extra** por cada cambio.

El resultado general



Teorema (Driscoll, Sarnak, Sleator, Tarjan, 1989)

Cualquier estructura de datos sobre la máquina de punteros en la que a lo mucho $p = O(1)$ punteros apuntan a un mismo nodo (en cualquier versión) se puede volver **parcialmente persistente** con $O(1)$ **overhead multiplicativo amortizado** en tiempo, y $O(1)$ **espacio extra** por cada cambio.

Es decir: si la operación original era $O(1)$, la versión persistente sigue siendo $O(1)$ amortizado — ¡casi gratis!

La técnica: nodos gordos (*fat nodes*)



La idea

Cada nodo guarda, además de sus campos originales, un **registro de modificaciones**: una lista de tuplas (campo, valor nuevo, tiempo) de tamaño acotado (hasta $2p$ entradas).

La técnica: nodos gordos (*fat nodes*)



La idea

Cada nodo guarda, además de sus campos originales, un **registro de modificaciones**: una lista de tuplas (campo, valor nuevo, tiempo) de tamaño acotado (hasta $2p$ entradas).

Leer un campo en la versión t

Se revisa el registro de modificaciones de más reciente a más antigua, y se toma la **primera** con tiempo $\leq t$; si no hay ninguna, se usa el valor original del nodo. $O(1)$, pues el registro tiene tamaño acotado.

¿Qué pasa cuando el registro se llena?



El problema

Si un nodo ya tiene $2p$ modificaciones registradas, no hay espacio para una más.

¿Qué pasa cuando el registro se llena?



El problema

Si un nodo ya tiene $2p$ modificaciones registradas, no hay espacio para una más.

La solución: *split*

Se crea un nodo **nuevo**, “limpio”, con todos los campos en su valor más reciente (registro vacío). Se redirigen hacia el nodo nuevo los p punteros que apuntaban al nodo viejo — **ahí es donde se usa** que a lo mucho p nodos apuntan a cualquier nodo.

Análisis amortizado: repaso del método del potencial



Recordando de clases anteriores

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}),$$

donde c_i es el costo real de la operación i , y Φ es una función de **potencial** que nunca cae por debajo de su valor inicial.

Análisis amortizado: repaso del método del potencial



Recordando de clases anteriores

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}),$$

donde c_i es el costo real de la operación i , y Φ es una función de **potencial** que nunca cae por debajo de su valor inicial.

Potencial para nodos gordos

$$\Phi = \sum_{\text{nodos } v} (\text{entradas usadas en el registro de } v).$$

El análisis: escritura sin *split*



Caso 1: hay espacio en el registro

Costo real $c_i = O(1)$: se agrega una entrada. El potencial sube en 1 (una entrada más usada): $\Delta\Phi = +1$.

El análisis: escritura sin *split*



Caso 1: hay espacio en el registro

Costo real $c_i = O(1)$: se agrega una entrada. El potencial sube en 1 (una entrada más usada): $\Delta\Phi = +1$.

Costo amortizado

$$\hat{c}_i = O(1) + 1 = O(1).$$

El análisis: escritura con *split*



Caso 2: el registro está lleno, se hace *split*

Costo real: $O(1)$ para el nodo nuevo, más $O(p)$ para redirigir los p punteros entrantes (cada redirección es, a su vez, una escritura de $O(1)$ en el predecesor correspondiente).

El análisis: escritura con *split*



Caso 2: el registro está lleno, se hace *split*

Costo real: $O(1)$ para el nodo nuevo, más $O(p)$ para redirigir los p punteros entrantes (cada redirección es, a su vez, una escritura de $O(1)$ en el predecesor correspondiente).

Cambio de potencial

El nodo viejo pasa de $2p$ entradas usadas a 0 ($\Delta\Phi_{\text{split}} = -2p$); cada una de las p redirecciones agrega a lo mucho 1 entrada en el predecesor correspondiente ($\Delta\Phi_{\text{redirect}} \leq +p$).

El análisis: escritura con *split*



Caso 2: el registro está lleno, se hace *split*

Costo real: $O(1)$ para el nodo nuevo, más $O(p)$ para redirigir los p punteros entrantes (cada redirección es, a su vez, una escritura de $O(1)$ en el predecesor correspondiente).

Cambio de potencial

El nodo viejo pasa de $2p$ entradas usadas a 0 ($\Delta\Phi_{\text{split}} = -2p$); cada una de las p redirecciones agrega a lo mucho 1 entrada en el predecesor correspondiente ($\Delta\Phi_{\text{redirect}} \leq +p$).

Costo amortizado

$$\hat{c}_i = O(p) + (-2p + p) = O(p) - p = O(1),$$

pues $p = O(1)$ por hipótesis.

¡Hazlo tú mismo!



Ejercicio

Implementa un **árbol binario de búsqueda** (BST) con persistencia parcial, usando la técnica de **nodos gordos**.

¡Hazlo tú mismo!



Ejercicio

Implementa un **árbol binario de búsqueda** (BST) con persistencia parcial, usando la técnica de **nodos gordos**.

Pista: en un BST, cada nodo tiene a lo mucho $p = 1$ padre apuntándolo (más, quizás, un puntero externo a la raíz de cada versión) — el caso más simple posible de aplicar el teorema.

Persistencia Total



¿Qué cambia respecto a la persistencia parcial?

Ahora se puede actualizar **cualquier** versión, no solo la más reciente. Las versiones ya **no** están ordenadas linealmente: forman un **árbol**.

Persistencia Total



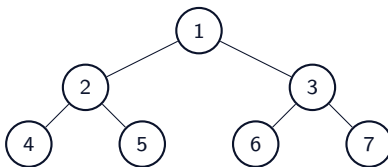
¿Qué cambia respecto a la persistencia parcial?

Ahora se puede actualizar **cualquier** versión, no solo la más reciente. Las versiones ya **no** están ordenadas linealmente: forman un **árbol**.

¿Por qué esto rompe la técnica anterior?

Los nodos gordos guardaban modificaciones ordenadas **solo hacia adelante en el tiempo**. Si se puede actualizar una versión “vieja”, necesitamos poder navegar **en ambas direcciones** del árbol de versiones.

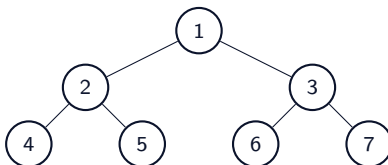
La idea: linearizar el árbol de versiones



Recorrido de Euler

Recorremos el árbol de versiones con un **recorrido de Euler**: al entrar a la versión i marcamos el tiempo (i ("se hace el cambio i ")), al salir marcamos i ("se deshace el cambio i ").

La idea: linearizar el árbol de versiones



Recorrido de Euler

Recorremos el árbol de versiones con un **recorrido de Euler**: al entrar a la versión i marcamos el tiempo $(i$ ("se hace el cambio i "), al salir marcamos $)i$ ("se deshace el cambio i ").

Ejemplo (subárbol de 1): $(1 (2 (4)_4 (5)_5)_2 (3 \dots$

Representando el tiempo



¿Para qué sirve esta secuencia de paréntesis?

La versión i “acumula” exactamente los cambios cuyo tiempo de apertura ($_i$ ocurre **antes o en** $_i$, y que **no** se cerraron antes de $_i$ — es decir, los cambios de sus **ancestros** en el árbol de versiones.

Representando el tiempo



¿Para qué sirve esta secuencia de paréntesis?

La versión i “acumula” exactamente los cambios cuyo tiempo de apertura ($($ ocurre **antes o en** $($, y que **no** se cerraron antes de $($ — es decir, los cambios de sus **ancestros** en el árbol de versiones.

El truco

Guardamos estos tiempos en una **estructura de mantenimiento de orden** (*order-maintenance data structure*), que soporta comparar dos tiempos e insertar un tiempo nuevo entre otros dos, ambos en $O(1)$.

Nodos gordos, ahora en dos direcciones



Cada nodo guarda dos registros de modificaciones

Uno para leer versiones **posteriores** a la base del nodo, y otro para versiones **anteriores** — así se puede responder consultas sin importar si la versión pedida está “adelante” o “atrás” respecto al nodo base.

Nodos gordos, ahora en dos direcciones



Cada nodo guarda dos registros de modificaciones

Uno para leer versiones **posteriores** a la base del nodo, y otro para versiones **anteriores** — así se puede responder consultas sin importar si la versión pedida está “adelante” o “atrás” respecto al nodo base.

El mismo patrón de antes, más delicado

Cuando un registro se llena, se hace *split*: se crea un nodo nuevo con la versión “actual” consolidada como base, y se reconstruyen los registros usando las modificaciones que quedan a cada lado.

Análisis amortizado de persistencia total



El resultado

Con la técnica de nodos gordos bidireccionales y un tamaño de registro proporcional a p (el número de punteros entrantes), se obtiene el **mismo** $O(1)$ amortizado por operación que en persistencia parcial.

Análisis amortizado de persistencia total



El resultado

Con la técnica de nodos gordos bidireccionales y un tamaño de registro proporcional a p (el número de punteros entrantes), se obtiene el **mismo** $O(1)$ amortizado por operación que en persistencia parcial.

El argumento (mismo espíritu que antes)

Un potencial proporcional al número de entradas usadas en los registros paga el costo de cada *split*, exactamente como en el Caso 2 de persistencia parcial — la demostración completa es más delicada (hay que contabilizar ambas direcciones y la estructura de orden), pero sigue el mismo patrón. Ver Driscoll–Sarnak–Sleator–Tarjan (1989) para el detalle completo.

Persistencia confluyente: por qué es difícil



El problema

Al permitir **combinar** dos versiones arbitrarias, una misma versión puede depender de u actualizaciones distintas acumuladas por caminos diferentes del DAG.

Persistencia confluyente: por qué es difícil



El problema

Al permitir **combinar** dos versiones arbitrarias, una misma versión puede depender de u actualizaciones distintas acumuladas por caminos diferentes del DAG.

Consecuencia

En el peor caso, hay hasta 2^u formas distintas en que las actualizaciones pueden haberse combinado — las técnicas de nodos gordos, pensadas para un único “historial lineal o de árbol”, ya no bastan en general.

Persistencia funcional: nunca mutar



La idea

Una estructura es **funcional** (o *puramente funcional*) si **jamás modifica un nodo existente**: cada actualización crea únicamente nodos **nuevos**, y los nodos viejos permanecen intactos para siempre.

Persistencia funcional: nunca mutar



La idea

Una estructura es **funcional** (o *puramente funcional*) si **jamás modifica un nodo existente**: cada actualización crea únicamente nodos **nuevos**, y los nodos viejos permanecen intactos para siempre.

¿Por qué esto da persistencia “gratis”?

Como nada se sobrescribe, **cualquier** puntero que ya tenías a una versión anterior sigue siendo válido y sigue describiendo exactamente esa versión — no hace falta ningún registro de modificaciones ni ningún *split*.

Path Copying: la técnica central



La pregunta

Si nunca modificamos nodos, ¿cómo actualizamos algo sin copiar **toda** la estructura en cada operación?

Path Copying: la técnica central



La pregunta

Si nunca modificamos nodos, ¿cómo actualizamos algo sin copiar **toda** la estructura en cada operación?

La respuesta: *Path Copying*

Solo se copian los nodos que están en el **camino** desde la raíz hasta el punto que cambia; todo lo demás (los subárboles que **no** cambian) se **comparte** entre la versión vieja y la nueva, vía punteros.

Path Copying en la Práctica

Path Copying: idea general



En una estructura con altura h

Cada actualización copia a lo mucho $O(h)$ nodos (el camino raíz–destino) y cuesta $O(h)$ tiempo y espacio extra — el resto de la estructura, potencialmente $O(n)$ nodos, se comparte sin tocar.

Path Copying: idea general



En una estructura con altura h

Cada actualización copia a lo mucho $O(h)$ nodos (el camino raíz–destino) y cuesta $O(h)$ tiempo y espacio extra — el resto de la estructura, potencialmente $O(n)$ nodos, se comparte sin tocar.

Veamos tres ejemplos, del más simple al más rico: un **Stack**, un **Segment Tree**, y un **Trie**.

Ejemplo 1: Stack persistente



La observación clave

Un *stack* (pila) implementado como lista enlazada **ya** es un “camino” de un solo nodo de largo: no hay nada que copiar aparte del nodo nuevo.

Algoritmo 1: $\text{PUSH}(S, x)$ — persistente

nuevo $\leftarrow$ nodo con valor x y siguiente $\leftarrow S$;
devolver nuevo ; // S (la versión vieja) sigue intacta

Ejemplo 1: Stack persistente

La observación clave

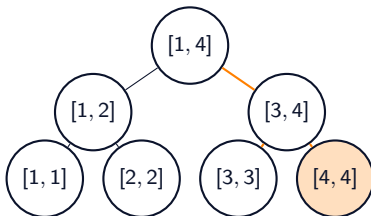
Un *stack* (pila) implementado como lista enlazada **ya** es un “camino” de un solo nodo de largo: no hay nada que copiar aparte del nodo nuevo.

Algoritmo 2: $\text{PUSH}(S, x)$ — persistente

nuevo $\leftarrow$ nodo con valor x y siguiente $\leftarrow S$;
devolver nuevo ; // S (la versión vieja) sigue intacta

PUSH es $O(1)$ tiempo y espacio, en cualquier nivel de persistencia — ¡ni siquiera necesitamos llamarlo *path copying*, es persistencia funcional en su forma más pura!

Ejemplo 2: Segment Tree persistente — estructura



Actualizar la posición 4

Solo los nodos en **naranja** (el camino raíz $\rightarrow [3, 4] \rightarrow [4, 4]$) se copian; $[1, 2]$, $[1, 1]$, $[2, 2]$, $[3, 3]$ se **comparten** con la versión anterior sin tocarlos.

Ejemplo 2: Segment Tree persistente — Update



Algoritmo 3: UPDATE(*nodo*, *l*, *r*, *pos*, *val*)

nuevo $\leftarrow$ copia de nodo ;

si $l = r$ entonces

 | nuevo.valor $\leftarrow$ *val* ;

en otro caso

 | $m \leftarrow \lfloor (l + r) / 2 \rfloor$;

 si $pos \leq m$ entonces

 | nuevo.izq $\leftarrow$ UPDATE(*nodo.izq*, *l*, *m*, *pos*, *val*) ;

 en otro caso

 | nuevo.der $\leftarrow$ UPDATE(*nodo.der*, $m+1$, *r*, *pos*, *val*) ;

 | nuevo.valor $\leftarrow$ combinar(nuevo.izq.valor, nuevo.der.valor) ;

devolver nuevo ;

Ejemplo 2: Segment Tree persistente — complejidad



Análisis

El segment tree tiene altura $h = O(\lg n)$: UPDATE copia exactamente un nodo por nivel, así que cada actualización cuesta $O(\lg n)$ tiempo y $O(\lg n)$ espacio nuevo.

Ejemplo 2: Segment Tree persistente — complejidad



Análisis

El segment tree tiene altura $h = O(\lg n)$: UPDATE copia exactamente un nodo por nivel, así que cada actualización cuesta $O(\lg n)$ tiempo y $O(\lg n)$ espacio nuevo.

Consultar una versión vieja

Basta guardar el puntero a la raíz de esa versión (devuelto por UPDATE en su momento) y hacer la consulta normal desde ahí: $O(\lg n)$, sin ningún costo adicional por ser una versión “del pasado”.

Ejemplo 2: Segment Tree persistente — complejidad



Análisis

El segment tree tiene altura $h = O(\lg n)$: UPDATE copia exactamente un nodo por nivel, así que cada actualización cuesta $O(\lg n)$ tiempo y $O(\lg n)$ espacio nuevo.

Consultar una versión vieja

Basta guardar el puntero a la raíz de esa versión (devuelto por UPDATE en su momento) y hacer la consulta normal desde ahí: $O(\lg n)$, sin ningún costo adicional por ser una versión “del pasado”.

Esta es exactamente la estructura conocida como **persistent segment tree**, muy usada en programación competitiva para responder consultas sobre prefijos o versiones históricas de un arreglo.

Ejemplo 3: Trie persistente

La misma idea, aplicada a un Trie

Al insertar una cadena s de largo L , solo se copian los $O(L)$ nodos a lo largo del camino que deletrea s desde la raíz; el resto del Trie se comparte.

Algoritmo 4: INSERT(nodo, s , i)

nuevo $\leftarrow$ copia de nodo ;

si $i = |s|$ **entonces**

 nuevo.esFinal $\leftarrow$ verdadero ;

$c \leftarrow s[i]$;

hijoViejo $\leftarrow$ nodo.hijos[c] (o un Trie vacío si no existe) ;

nuevo.hijos[c] $\leftarrow$ INSERT(hijoViejo, s , $i+1$) ;

devolver nuevo ;

Ejemplo 3: Trie persistente — complejidad



Análisis

INSERT copia exactamente un nodo por carácter de s : $O(L)$ tiempo y espacio nuevo, donde $L = |s|$.

Ejemplo 3: Trie persistente — complejidad



Análisis

INSERT copia exactamente un nodo por carácter de s : $O(L)$ tiempo y espacio nuevo, donde $L = |s|$.

¡Piénsalo!

Si insertas n cadenas de largo total $\sum L_i$, ¿cuánto espacio **total** usa el Trie persistente, comparado con guardar n copias completas del Trie?

Ejemplo 3: Trie persistente — complejidad



Análisis

INSERT copia exactamente un nodo por carácter de s : $O(L)$ tiempo y espacio nuevo, donde $L = |s|$.

¡Piénsalo!

Si insertas n cadenas de largo total $\sum L_i$, ¿cuánto espacio **total** usa el Trie persistente, comparado con guardar n copias completas del Trie?

$O(\sum L_i)$ en total — el mismo orden que un Trie **efímero** normal, porque cada carácter insertado crea a lo mucho un nodo nuevo, sin importar cuántas versiones acumules.

Comparando los tres ejemplos



Stack	Segment Tree	Trie

Comparando los tres ejemplos



	Stack	Segment Tree	Trie
Altura relevante	1	$O(\lg n)$	$O(L)$

Comparando los tres ejemplos



	Stack	Segment Tree	Trie
Altura relevante	1	$O(\lg n)$	$O(L)$
Nodos copiados por operación	1	$O(\lg n)$	$O(L)$

Comparando los tres ejemplos



	Stack	Segment Tree	Trie
Altura relevante	1	$O(\lg n)$	$O(L)$
Nodos copiados por operación	1	$O(\lg n)$	$O(L)$
Espacio nuevo por operación	$O(1)$	$O(\lg n)$	$O(L)$

Comparando los tres ejemplos



	Stack	Segment Tree	Trie
Altura relevante	1	$O(\lg n)$	$O(L)$
Nodos copiados por operación	1	$O(\lg n)$	$O(L)$
Espacio nuevo por operación	$O(1)$	$O(\lg n)$	$O(L)$

En los tres casos: **path copying** cuesta exactamente el largo del camino que cambia, sin importar qué tan grande sea el resto de la estructura.

Resumen de la sesión



- ▶ Sobre la **máquina de punteros**, cualquier estructura con a lo mucho $p = O(1)$ punteros entrantes por nodo se puede volver **parcialmente persistente** en $O(1)$ amortizado, vía **nodos gordos** (registro de modificaciones + *split* cuando se llena).
- ▶ La **persistencia total** logra el mismo $O(1)$ amortizado, con nodos gordos **bidireccionales** sobre un árbol de versiones linearizado (recorrido de Euler + estructura de mantenimiento de orden).
- ▶ La **persistencia confluyente** es fundamentalmente más difícil (hasta 2^u combinaciones posibles); la **funcional** evita el problema por completo: nunca modificar, solo crear.
- ▶ **Path copying** implementa persistencia funcional copiando solo el camino que cambia: $O(1)$ en un Stack, $O(\lg n)$ en un Segment Tree, $O(L)$ en un Trie.

Gracias

Referencias



Bibliografía citada

- ▶ Driscoll, J. R., Sarnak, N., Sleator, D. D., & Tarjan, R. E. (1989). Making data structures persistent. *Journal of Computer and System Sciences*, 38(1).
- ▶ Cormen, Thomas H. et al. (2022). *Introduction to Algorithms*. 4th ed. The MIT Press.
- ▶ Demaine, Erik (2021). *6.851: Advanced Data Structures (Spring'21)*. <https://courses.csail.mit.edu/6.851/spring21/>.

Agradecimientos



Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.

Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.